Processes, scheduling and interrupts



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Processes and scheduling



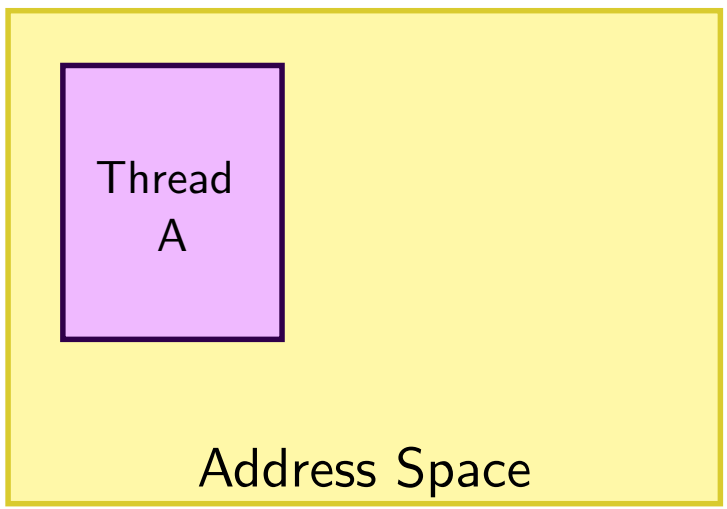
Process, thread?

- Confusion about the terms *process*, *thread* and *task*
- In UNIX, a process is created using `fork()` and is composed of
  - An address space, which contains the program code, data, stack, shared libraries, etc.
  - A single thread, which is the only entity known by the scheduler.
- Additional threads can be created inside an existing process, using `pthread_create()`
  - They run in the same address space as the initial thread of the process
  - They start executing a function passed as argument to `pthread_create()`

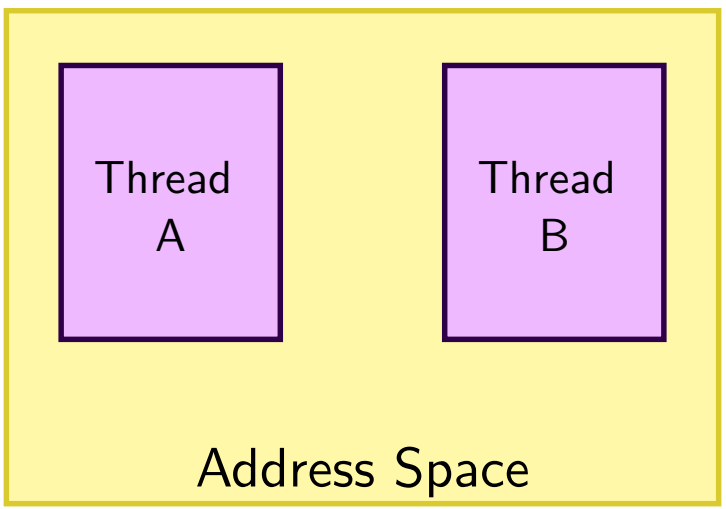


Process, thread: kernel point of view

- In kernel space, each thread running in the system is represented by a structure of type struct task_struct
- From a scheduling point of view, it makes no difference between the initial thread of a process and all additional threads created dynamically using `pthread_create()`



Process after fork()



Same process after pthread_create()

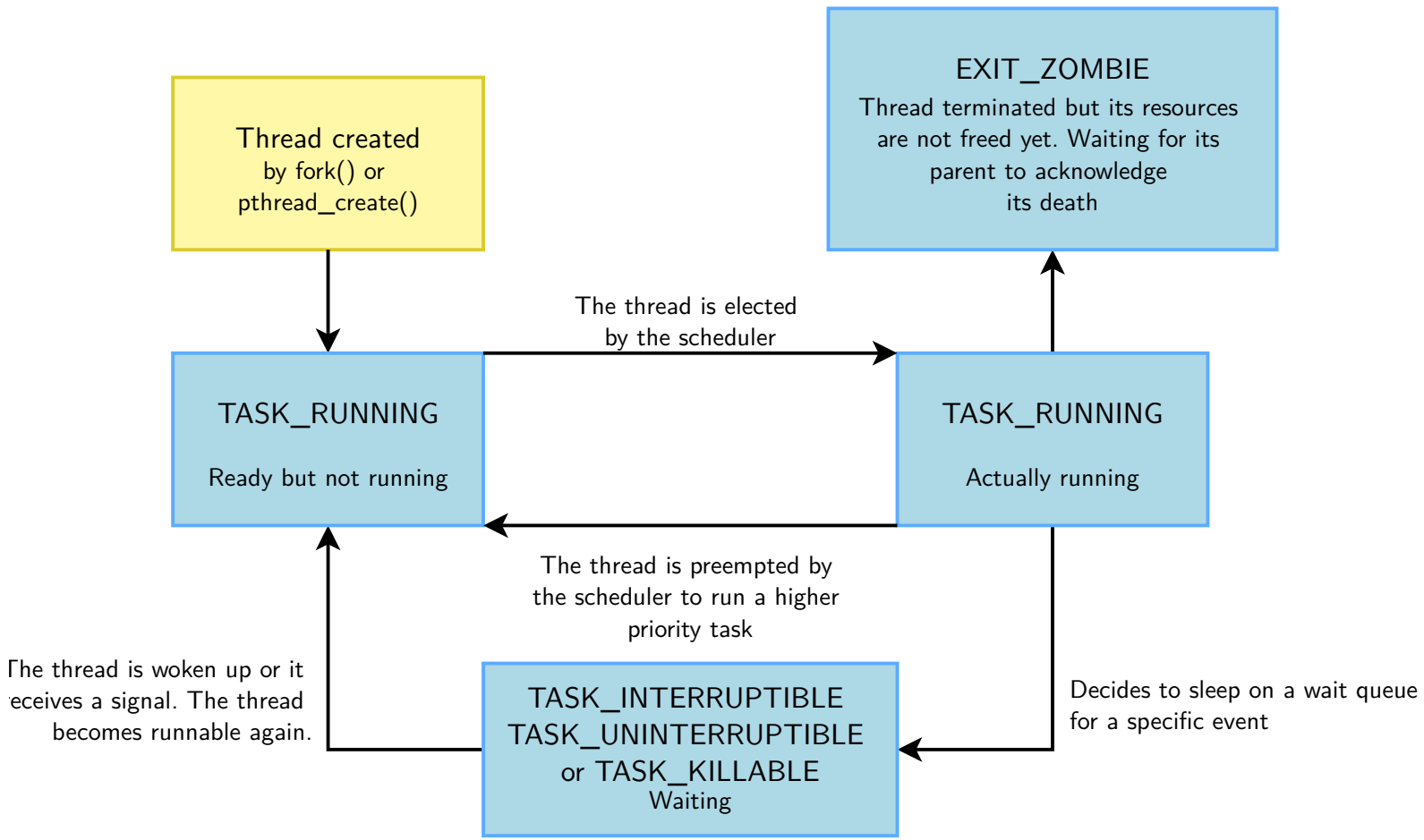


Relation between execution mode, address space and context

- When speaking about *process* and *thread*, these concepts need to be clarified:
  - *Mode* is the level of privilege allowing to perform some operations:
    - *Kernel Mode*: in this level CPU can perform any operation allowed by its architecture; any instruction, any I/O operation, any area of memory accessed.
    - *User Mode*: in this level, certain instructions are not permitted (especially those that could alter the global state of the machine), some memory areas cannot be accessed.
  - Linux splits its *address space* in *kernel space* and *user space*
    - *Kernel space* is reserved for code running in *Kernel Mode*.
    - *User space* is the place were applications execute (accessible from *Kernel Mode*).
  - The *context* determines what operations are allowed (mostly about sleepability)
    - In *process context*, most operations are allowed, it is the most permissive context
    - *Atomic context* covers many possible execution environments (Interrupt context, softirq context, NMI context, spinlock protected regions...) where sleeping is not allowed or even not possible at all.

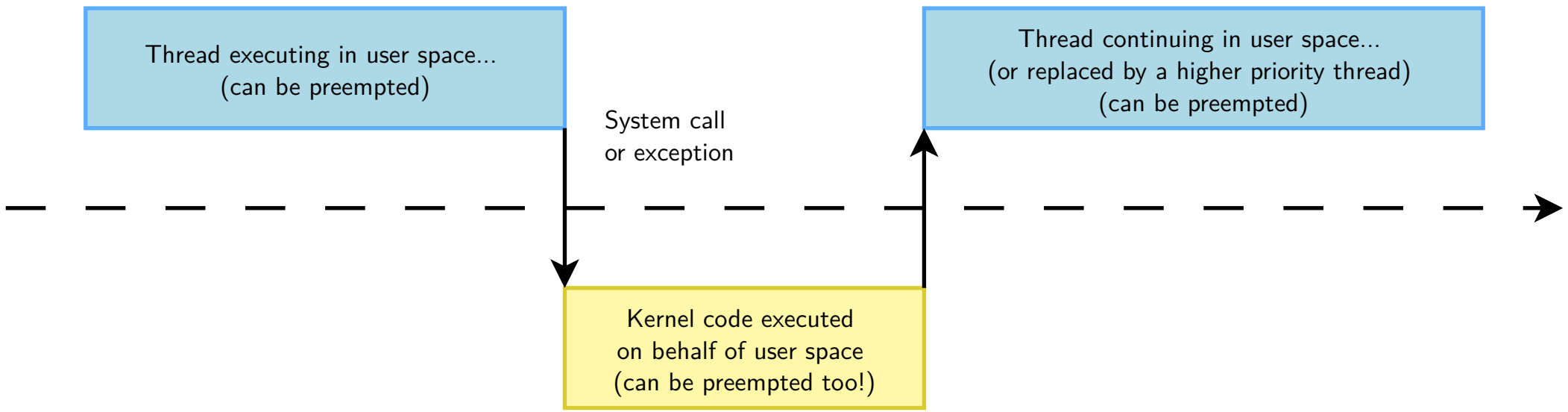


A thread life





Execution of system calls

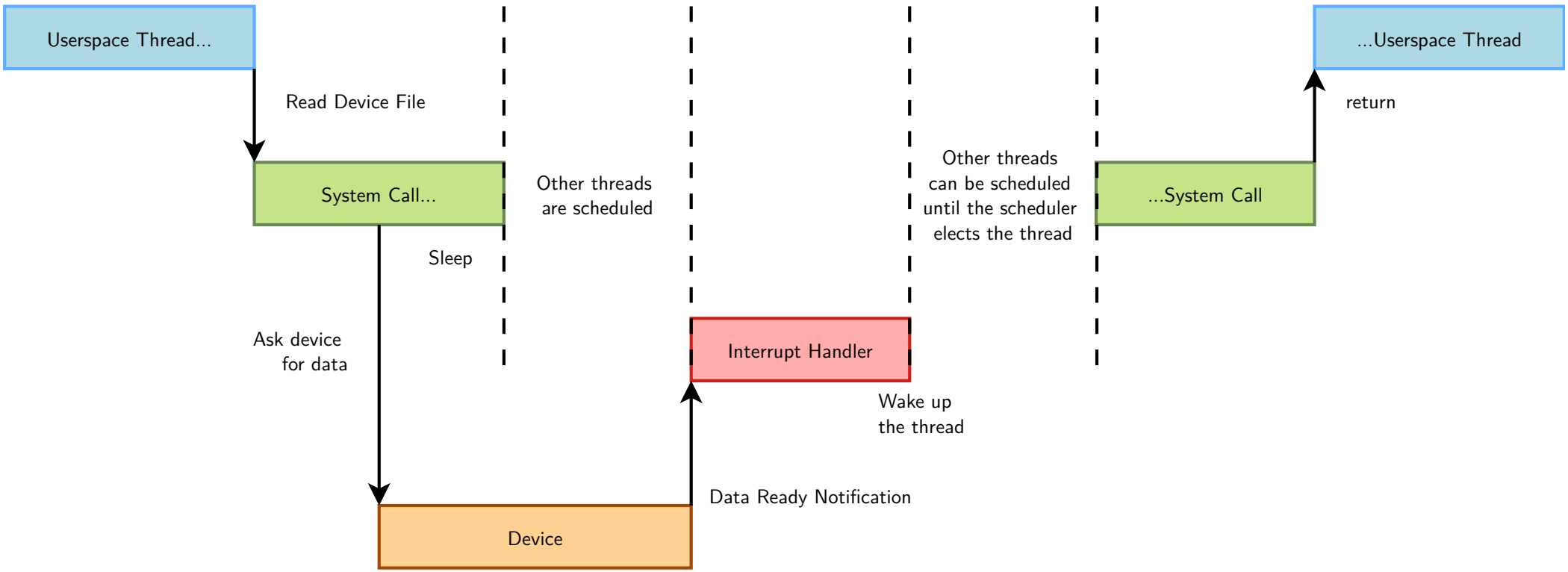


The execution of system calls takes place in the context of the thread requesting them.



Sleeping

Sleeping



Sleeping is needed when a userspace or kernelspace thread is waiting for data.



How to sleep with a wait queue 1/3

- Must declare a wait queue, which will be used to store the list of threads waiting for an event
- Dynamic queue declaration:
  - Typically one queue per device managed by the driver
  - It's convenient to embed the wait queue inside a per-device data structure.
  - Example from drivers/net/ethernet/marvell/mvmdio.c:

```
struct orion_mdio_dev {
        ...
        wait_queue_head_t smi_busy_wait;
};
struct orion_mdio_dev *dev;
...
init_waitqueue_head(&dev->smi_busy_wait);
```

- Static queue declaration:
  - Using a global variable when a global resource is sufficient
  - `DECLARE_WAIT_QUEUE_HEAD(module_queue);`



How to sleep with a wait queue 2/3

Several ways to make a thread sleep

- `void wait_event(queue, condition);`
  - Sleeps until the wait queue is woken up **and** the given C expression is true. Caution: once entered, the thread sleep cannot be interrupted!
- `int wait_event_killable(queue, condition);`
  - Can be interrupted, but only by a *fatal* signal (SIGKILL). Returns -ERESTARTSYS if interrupted.
- `int wait_event_interruptible(queue, condition);`
  - The most common variant
  - Can be interrupted by any signal. Returns -ERESTARTSYS if interrupted.



How to sleep with a wait queue 3/3

- `int wait_event_timeout(queue, condition, timeout);`
  - Also stops sleeping when the thread is woken up **or** the timeout expired (a timer is used).
  - Returns `0` if the timeout elapsed, non-zero if the condition was met.
- `int wait_event_interruptible_timeout(queue, condition, timeout);`
  - Same as above, interruptible.
  - Returns `0` if the timeout elapsed, `-ERESTARTSYS` if interrupted, positive value if the condition was met.



How to sleep with a wait queue - Example

```
sig = wait_event_interruptible(ibmvtpm->wq,
                               !ibmvtpm->tpm_processing_cmd);
if (sig)
        return -EINTR;
```

From drivers/char/tpm/tpm_ibmvtpm.c



Waking up!

Typically done by interrupt handlers when data sleeping threads are waiting for become available.

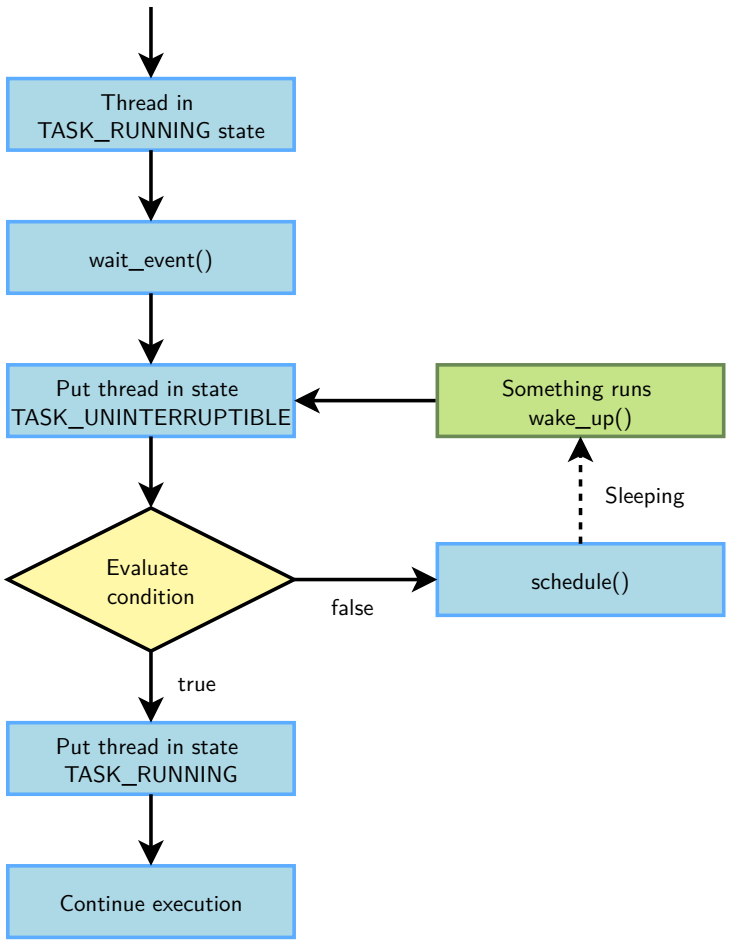
- `wake_up(&queue);`
  - Wakes up all threads in the wait queue
- `wake_up_interruptible(&queue);`
  - Wakes up all threads waiting in an interruptible sleep on the given queue



Exclusive vs. non-exclusive

- wait_event_interruptible() puts a thread in a non-exclusive wait.
  - All non-exclusive threads are woken up by wake_up() / wake_up_interruptible()
- wait_event_interruptible_exclusive() puts a thread in an exclusive wait.
  - wake_up() / wake_up_interruptible() wakes up all non-exclusive threads and only one exclusive thread
  - wake_up_all() / wake_up_interruptible_all() wakes up all non-exclusive and all exclusive threads
- Exclusive sleeps are useful to avoid waking up multiple threads when only one will be able to *consume* the event.
- Non-exclusive sleeps are useful when the event can *benefit* to multiple threads.

Sleeping and waking up - Implementation



The scheduler doesn't keep evaluating the sleeping condition!

- `wait_event(queue, cond);`
  The thread is put in the TASK_UNINTERRUPTIBLE state.
- `wake_up(&queue);`
  All threads waiting in `queue` are woken up, so they get scheduled later and have the opportunity to evaluate the condition again and go back to sleep if it is not met.

See include/linux/wait.h for implementation details.



How to sleep with completions 1/2

- Use wait_for_completion() when no particular condition must be enforced at the time of the wake-up
  - Leverages the power of wait queues
  - Simplifies its use
  - Highly efficient using low level scheduler facilities
- Preparation of the completion structure:
  - Static declaration and initialization:
    ```
    DECLARE_COMPLETION(setup_done);
    ```
  - Dynamic declaration:
    ```
    init_completion(&object->setup_done);
    ```
  - The completion object should get a meaningful name (eg. not just “done”).
- Ready to be used by signal consumers and providers as soon as the completion object is initialized
- See include/linux/completion.h for the full API
- Internal documentation at scheduler/completion



How to sleep with completions 2/2

- Enter a wait state with `void wait_for_completion(struct completion *done)`
  - All wait_event() flavors are also supported, such as: wait_for_completion_timeout(), wait_for_completion_interruptible() / _timeout(), wait_for_completion_killable() / _timeout(), etc.
- Wake up consumers with `void complete(struct completion *done)`
  - Several calls to complete() are valid, they will wake up the same number of threads waiting on this object (acts as a FIFO).
  - A single complete_all() call would wake up all present and future threads waiting on this completion object
- Reset the counter with `void reinit_completion(struct completion *done)`
  - Resets the number of "done" completions still pending
  - Mind not to call init_completion() twice, which could confuse the enqueued threads



Blocking

- Use helpers which implement software loops or use hardware timers
  - udelay() waste CPU cycles in order to save a couple of context switches, suitable for ≤ 10us or in atomic situations
  - usleep()/usleep_range()/msleep() put the thread in sleep for a given amount of micro/milliseconds (not suitable in atomic contexts)
  - If in doubt, use fsleep(), which will use the more suitable internal function depending on the period you've asked!



Waiting when hardware is involved

- When hardware is involved in the waiting process
  - but there is no interrupt available
  - or because a context switch would be too expensive
- Specific polling I/O accessors may be used:
  - Exhaustive list in include/linux/iopoll.h

```
int read[bwlq]_poll_timeout_[atomic](addr, val, cond,
                                     delay_us, timeout_us)
```

    - `addr`: I/O memory location
    - `val`: Content of the register pointed with
    - `cond`: Boolean condition based on `val`
    - `delay_us`: Polling delay between reads
    - `timeout_us`: Timeout delay after which the operation fails and returns -ETIMEDOUT
  - `_atomic` variant uses udelay() instead of usleep().
- Avoid implementing custom busy-wait loops if possible